

Lekcja 13: Bazy Danych i Podstawy SQL

#lekcja #python #bazy_danych #sql #dbms

Witaj w kolejnej lekcji! Do tej pory nauczyliśmy się, jak pracować z plikami, takimi jak CSV czy JSON. To świetne rozwiązania do przechowywania mniejszych ilości danych. Co jednak, gdy nasza aplikacja rośnie i potrzebujemy przechowywać, zarządzać i odpytywać tysiące, a nawet miliony rekordów w sposób wydajny i bezpieczny? Właśnie tutaj z pomocą przychodzą bazy danych.

W tej lekcji poznamy podstawowe pojęcia związane z bazami danych, nauczymy się języka SQL do komunikacji z nimi i zobaczymy, jak to wszystko połączyć z Pythonem.

1. Czym jest Baza Danych?

Zacznijmy od absolutnych podstaw. Co kryje się pod pojęciem "baza danych"?

Definition

Baza danych (ang. database) to uporządkowany zbiór informacji (danych) zapisanych w formie elektronicznej w systemie komputerowym. Pozwala na łatwe przechowywanie, modyfikowanie, usuwanie i wyszukiwanie danych.

Wyobraź sobie ogromną, inteligentną szafkę z kartotekami. Każda szuflada to **tabela**, każda kartoteka w szufladzie to **rekord** (np. dane jednego użytkownika), a każde pole na kartotece (imię, nazwisko, wiek) to **kolumna**. Baza danych to właśnie taka szafka, ale w wersji cyfrowej.

Rodzaje Baz Danych

Istnieje wiele typów baz danych, ale na początek warto znać trzy podstawowe grupy:

1. **Relacyjne (SQL):** To najpopularniejszy typ. Dane są w nich zorganizowane w tabelach, które składają się z wierszy i kolumn. Tabele mogą być ze sobą powiązane (stąd nazwa "relacyjne"). Do komunikacji z nimi używamy języka SQL. Przykłady: *PostgreSQL*, *MySQL*, *SQLite*.
2. **Nierelacyjne (NoSQL):** Bardziej elastyczne bazy, które pozwalają przechowywać dane w różnych formatach, np. dokumentów (jak w JSON), klucz-wartość czy grafów. Są świetne do danych nieustrukturyzowanych. Przykłady: *MongoDB*, *Redis*, *Cassandra*.
3. **OLTP (On-Line Transaction Processing):** To specjalistyczne bazy danych zoptymalizowane pod kątem obsługi dużej liczby krótkich transakcji, np. w systemach

bankowych czy rezerwacyjnych.

Note

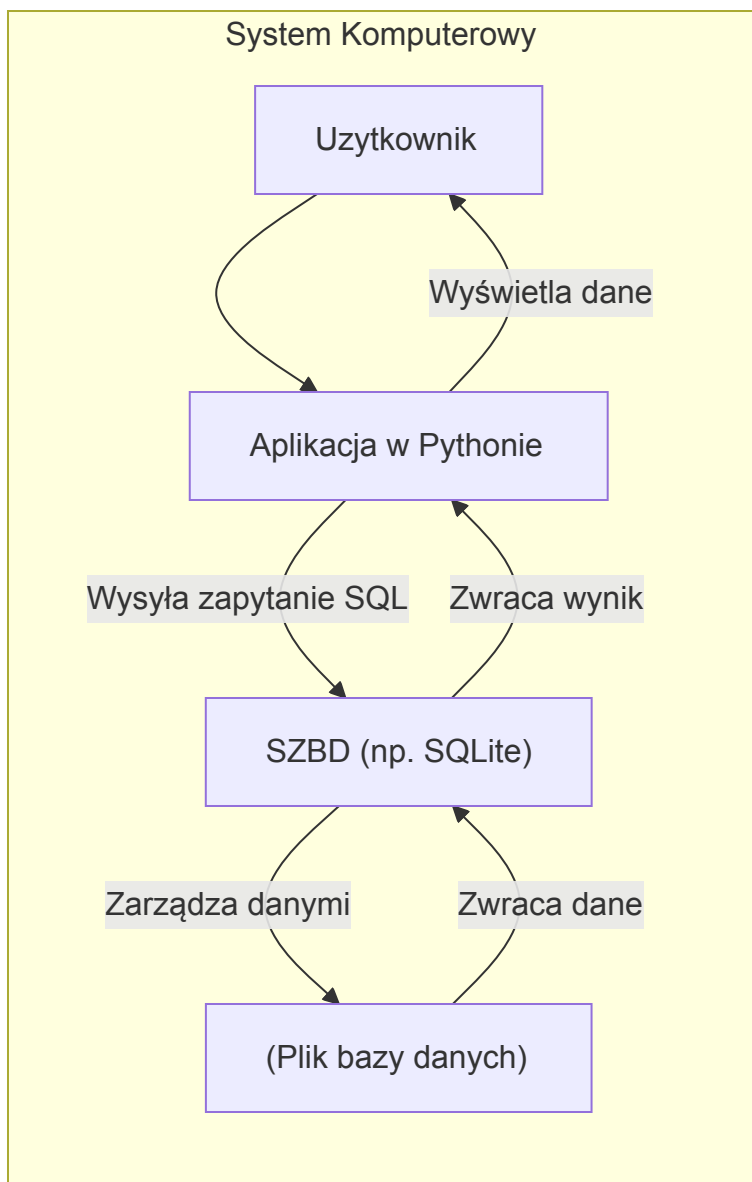
W tej lekcji skupimy się na relacyjnych bazach danych, ponieważ są one fundamentem i doskonałym punktem wyjścia do nauki pracy z danymi.

2. System Zarządzania Bazą Danych (SZBD/DBMS)

Sama baza danych to tylko zbiór danych. Aby móc z nią pracować – dodawać nowe informacje, wyszukiwać, aktualizować – potrzebujemy specjalnego oprogramowania.

Definition

System Zarządzania Bazą Danych (SZBD, ang. Database Management System, DBMS) to oprogramowanie, które działa jako pośrednik między użytkownikiem (lub aplikacją) a fizyczną bazą danych. Jego głównym zadaniem jest zarządzanie danymi, dbanie o ich spójność, bezpieczeństwo oraz wykonywanie operacji.



Popularne SZBD:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- **SQLite**
- Oracle Database

Tip

W naszym kursie będziemy używać SQLite. Jest to "lekki" system bazodanowy, który jest wbudowany w Pythona (nie trzeba nic instalować!). Zapisuje całą bazę danych w jednym pliku, co czyni go idealnym narzędziem do nauki i tworzenia mniejszych aplikacji.

3. Język Zapytań SQL – Podstawy

Skoro mamy już bazę i system do zarządzania nią, potrzebujemy jeszcze języka, aby się z nimi komunikować. Tym językiem jest SQL.

Info

SQL (Structured Query Language) to standardowy język używany do zarządzania danymi w relacyjnych bazach danych. Pozwala na tworzenie tabel, wstawianie, aktualizowanie, usuwanie i pobieranie danych.

W Pythonie do pracy z bazą SQLite użyjemy wbudowanego modułu `sqlite3`.

Nawiązywanie połączenia

Najpierw musimy połączyć się z naszą bazą danych. Jeśli plik bazy nie istnieje, zostanie automatycznie utworzony.

```
import sqlite3

# Nawiąż połączenie z bazą danych (plik 'kurs.db' zostanie utworzony)
# Zmienna 'conn' to obiekt reprezentujący połączenie
conn = sqlite3.connect('kurs.db')

# Aby wykonywać polecenia SQL, potrzebujemy obiektu 'kursora'
# Cursor to obiekt, który pozwala na przemieszczanie się po rekordach i
# wykonywanie zapytań
c = conn.cursor()

print("Połączono z bazą danych!")

# Na koniec ZAWSZE zamykamy połączenie
conn.close()
```

CREATE TABLE – Tworzenie tabeli

Stwórzmy naszą pierwszą tabelę, w której będziemy przechowywać informacje o miastach.

```
import sqlite3

conn = sqlite3.connect('kurs.db')
c = conn.cursor()
```

```
# Polecenie execute() wykonuje zapytanie SQL
# Używamy potrójnych cudzysłowów dla czytelności wielolinowych zapytań
c.execute('''
    CREATE TABLE IF NOT EXISTS miasta (
        id INTEGER PRIMARY KEY,
        nazwa TEXT NOT NULL,
        populacja INTEGER
    )
''')

# Zatwierdzamy zmiany w bazie danych
conn.commit()

print("Tabela 'miasta' została utworzona.")

conn.close()
```

Analiza zapytania CREATE TABLE :

- CREATE TABLE miasta : Stwórz tabelę o nazwie miasta .
- IF NOT EXISTS : Opcjonalny dodatek, który zapobiega błędowi, jeśli tabela już istnieje.
- id INTEGER PRIMARY KEY : Stwórz kolumnę id typu INTEGER (liczba całkowita), która będzie **kluczem głównym** (unikalny identyfikator każdego wiersza).
- nazwa TEXT NOT NULL : Stwórz kolumnę nazwa typu TEXT (tekst), która nie może być pusta.
- populacja INTEGER : Stwórz kolumnę populacja typu INTEGER .

INSERT INTO – Wstawianie danych

Teraz, gdy mamy tabelę, wstawmy do niej kilka rekordów.

```
import sqlite3

conn = sqlite3.connect('kurs.db')
c = conn.cursor()

# Sposób 1: Bezpieczne wstawianie danych za pomocą symboli zastępczych (?)
c.execute("INSERT INTO miasta (nazwa, populacja) VALUES (?, ?)",
    ('Warszawa', 1794166))
c.execute("INSERT INTO miasta (nazwa, populacja) VALUES (?, ?)", ('Kraków',
    779996))

# Sposób 2: Wstawianie wielu rekordów naraz
miasta_do_dodania = [
```

```
        ('Łódź', 679941),
        ('Wrocław', 642869),
        ('Poznań', 534813)
    ]
    c.executemany("INSERT INTO miasta (nazwa, populacja) VALUES (?, ?)",
        miasta_do_dodania)

    conn.commit()
    print(f"Dodano {c.rowcount * len(miasta_do_dodania)} rekordy do tabeli 'miasta'.")

    conn.close()
```

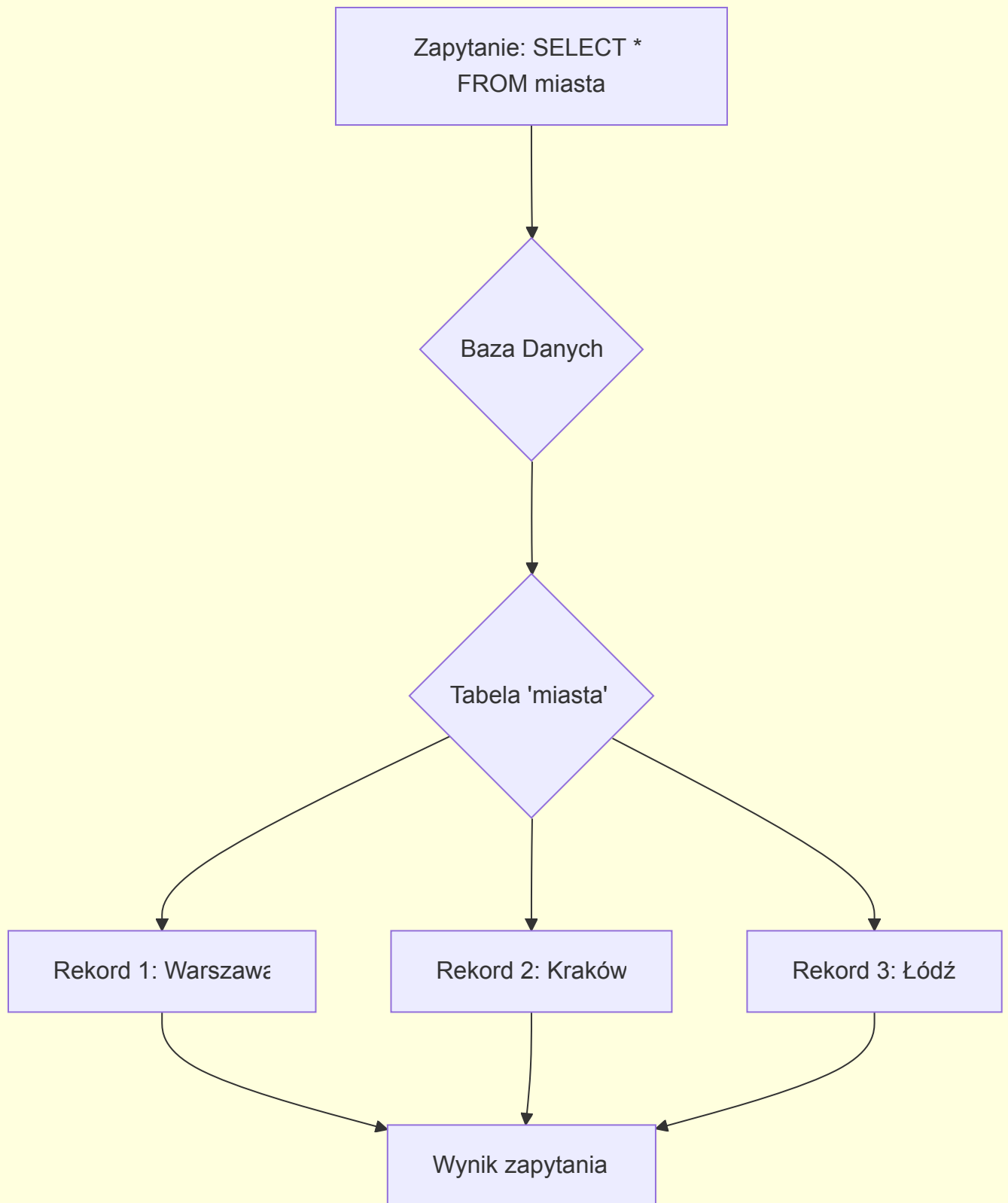
Tip

Zawsze używaj symboli zastępczych (?) w SQLite) do wstawiania danych do zapytań. Chroni to Twoją bazę przed atakami typu SQL Injection!

SELECT – Pobieranie danych

Pobierzmy dane, które właśnie wstawiliśmy.

Proces SELECT



```
import sqlite3
```

```
conn = sqlite3.connect('kurs.db')
```

```

c = conn.cursor()

# Pobierz wszystkie kolumny (*) z tabeli 'miasta'
c.execute("SELECT * FROM miasta")

# fetchall() pobiera wszystkie pasujące rekordy
wszystkie_miasta = c.fetchall()
print("Wszystkie miasta w bazie:")
for miasto in wszystkie_miasta:
    print(miasto)

print("\nMiasta z populacją powyżej 700 000:")
# Możemy filtrować wyniki za pomocą klauzuli WHERE
c.execute("SELECT nazwa, populacja FROM miasta WHERE populacja > ?",
(700000,))

# fetchone() pobiera jeden pasujący rekord
miasta_powyzej_700k = c.fetchall()
for miasto in miasta_powyzej_700k:
    print(f" - {miasto[0]}, populacja: {miasto[1]}")

conn.close()

```

UPDATE – Aktualizacja danych

Założmy, że populacja Poznania wzrosła. Zaktualizujmy ten rekord.

```

import sqlite3

conn = sqlite3.connect('kurs.db')
c = conn.cursor()

nowa_populacja = 540000
miasto_do_aktualizacji = 'Poznań'

c.execute("UPDATE miasta SET populacja = ? WHERE nazwa = ?",
(nowa_populacja, miasto_do_aktualizacji))

conn.commit()
print(f"Zaktualizowano populację dla miasta: {miasto_do_aktualizacji}.
Zmieniono {c.rowcount} rekordów.")

# Sprawdźmy, czy zmiana została zapisana
c.execute("SELECT * FROM miasta WHERE nazwa = ?", (miasto_do_aktualizacji,))
zaktualizowane_miasto = c.fetchone()

```



```
print(f"Nowe dane: {zaktualizowane_miasto}")
```

```
conn.close()
```

Zadania do samodzielnej pracy

Teraz Twoja kolej! Użyj modułu `sqlite3` i wiedzy z lekcji, aby rozwiązać poniższe zadania. Stwórz nowy plik `.py` dla każdego zadania.

Zadania proste

1. Zadanie 1 – Stwórz tabelę książek

Napisz skrypt, który połączy się z bazą `biblioteka.db` i stworzy w niej tabelę `ksiazki`. Tabela powinna mieć następujące kolumny:

- `id` (INTEGER, klucz główny)
- `tytul` (TEXT, nie może być pusty)
- `autor` (TEXT, nie może być pusty)
- `rok_wydania` (INTEGER)

2. Zadanie 2 – Dodaj książki

Napisz skrypt, który doda do tabeli `ksiazki` (stworzonej w zadaniu 1) trzy dowolne książki. Użyj metody `executemany` do dodania wszystkich książek za jednym razem.

3. Zadanie 3 – Wyświetl całą bibliotekę

Napisz skrypt, który pobierze i wyświetli w konsoli wszystkie książki (wszystkie kolumny) z tabeli `ksiazki`.

4. Zadanie 4 – Wyszukaj książki autora

Napisz skrypt, który pobierze i wyświetli tylko te książki z tabeli `ksiazki`, które zostały napisane przez Twojego ulubionego autora.

5. Zadanie 5 – Zaktualizuj rok wydania

Wybierz jedną z dodanych książek i napisz skrypt, który zaktualizuje jej `rok_wydania` na inną wartość. Po aktualizacji wyświetl dane tej książki, aby potwierdzić, że zmiana się powiodła.

Zadania z gwiazdką (challenge)

6. Zadanie 6 – Dwie tabele: Studenci i Audytoria

Napisz skrypt, który w nowej bazie `uczelnia.db` stworzy dwie tabele:

- studenci z kolumnami: `id_studenta` (klucz główny), `imie` (TEXT), `nazwisko` (TEXT).
- audytoria z kolumnami: `id_audytorium` (klucz główny), `nazwa_budynku` (TEXT), `numer_sali` (INTEGER).

7. 🧠 Zadanie 7 – Wypełnij dane uczelni

Napisz skrypt, który wypełni tabele studenci i audytoria przykładowymi danymi. Dodaj co najmniej 4 studentów i 3 audytoria.

8. 🧠 Zadanie 8 – Połącz tabele (Relacja)

To zadanie wprowadza kluczowe pojęcie relacji. Chcemy przypisać studentów do audytoriów (np. na egzamin). Aby to zrobić, stwórz trzecią tabelę o nazwie przypisania w tej samej bazie uczelnia.db. Tabela powinna mieć strukturę:

- `id_przypisania` (INTEGER, klucz główny)
- `id_studenta` (INTEGER) – będzie to tzw. **klucz obcy** wskazujący na `id_studenta` w tabeli `studenci`.
- `id_audytorium` (INTEGER) – **klucz obcy** wskazujący na `id_audytorium` w tabeli `audytoria`.

9. 🧠 Zadanie 9 – Dokonaj przypisań

Napisz skrypt, który dokona przypisań. Dla każdego studenta z tabeli `studenci` dodaj wpis do tabeli `przypisania`, łącząc go z jednym z audytoriów.

10. 🧠 Zadanie 10 – Funkcja wyszukująca z JOIN

Napisz funkcję w Pythonie `znajdz_sale_studenta(nazwisko)`, która przyjmuje nazwisko studenta jako argument. Funkcja powinna połączyć się z bazą, a następnie znaleźć i wyświetlić informację, w którym budynku i w jakiej sali znajduje się dany student.

Tip

Aby rozwiązać to zadanie, będziesz potrzebować klauzuli JOIN w zapytaniu SELECT. Pozwala ona łączyć wiersze z dwóch lub więcej tabel na podstawie powiązanych kolumn.

Przykład: `SELECT t1.kolumna, t2.kolumna FROM tabela1 AS t1 JOIN tabela2 AS t2 ON t1.id = t2.id_z_tabeli1;`